

2.1 Need of Python

Python is a **high-level, general-purpose programming language** widely used for a variety of applications ranging from web development to data science and artificial intelligence.

Why Python is Needed:

- **Easy to Learn & Readable:** Simple syntax that resembles English.
 - **Productivity & Speed:** Allows developers to write less code and accomplish more.
 - **Wide Application Areas:**
 - Web development (e.g., Django, Flask)
 - Data Science and Machine Learning (e.g., pandas, NumPy, scikit-learn)
 - Automation/Scripting
 - Game Development
 - Desktop GUI Applications
 - **Large Community Support** and Rich Ecosystem of Libraries.
 - **Cross-platform Compatibility**
-

2.2 History of Python

- **Created by:** Guido van Rossum
- **Initial Release:** 1991
- **Organization:** Python Software Foundation
- **Current Version** (as of June 2025): Python 3.12+

Evolution:

- Python 1.0: Released in 1991
 - Python 2.0: Released in 2000, introduced features like garbage collection
 - Python 3.0: Released in 2008, not backward-compatible with Python 2
-

2.3 Features and Limitations of Python

Features:

- **Simple & Easy to Learn**
- **Interpreted Language** (no compilation needed)
- **Dynamically Typed**
- **Extensive Standard Library**
- **Object-Oriented & Functional Programming Support**
- **Portable & Cross-platform**
- **Open Source and Free**
- **Ideal for Prototyping and Rapid Development**

Limitations:

- **Speed:** Slower than compiled languages like C/C++
 - **Mobile App Development:** Not preferred for mobile platforms
 - **Memory Usage:** Higher due to dynamic typing
 - **Runtime Errors:** Type-related errors are discovered at runtime
 - **Threading Limitations** due to the Global Interpreter Lock (GIL)
-

2.4 Python with Respect to Other Languages

Feature	Python	C	C++	Java	JavaScript
Level	High-level	Low-level	Mid-level	High-level	High-level
Typing	Dynamic	Static	Static	Static	Dynamic
Ease of Learning	Easy	Difficult	Moderate	Moderate	Easy
Speed	Moderate	Very Fast	Fast	Moderate	Fast
Memory Management	Automatic	Manual	Manual/Auto	Automatic	Automatic
Paradigm	Multi-paradigm	Procedural	Multi-paradigm	OOP	Multi-paradigm
Use Case	General purpose	System/Embedded	System, Games	Enterprise Apps	Web Interactivity
Compilation	Interpreted	Compiled	Compiled	Compiled to Bytecode	Interpreted

2.5 Top Python Implementations

Python is not just a language; it has several **implementations** (interpreters) optimized for different platforms and use-cases.

Implementation	Description
----------------	-------------

CPython	The default and most widely used implementation written in C.
Jython	Python implemented in Java, runs on the JVM.
IronPython	Python implemented in C#, runs on the .NET framework.
PyPy	Fast, JIT-compiled implementation of Python. Often faster than CPython.
MicroPython	A lean Python implementation for microcontrollers and embedded systems.
Brython	Python for the browser, compiles Python to JavaScript.

How does a Python Interpreter Works?

A **Python interpreter** is a program that reads and executes Python code **line-by-line** (i.e., **interprets** it). It performs several steps internally to convert human-readable code into machine-executable instructions.

Step-by-Step Working of Python Interpreter

Let's walk through the key stages:

1. Source Code (.py file)

You write Python code like this:

```
x = 5
print(x + 2)
```

2. Lexical Analysis (Tokenizer / Lexer)

- Python breaks the source code into **tokens** (identifiers, keywords, literals, operators, etc.).
- For example, `x`, `=`, `5`, `print`, `(`, `x`, `+`, `2`, `)`.

3. Parsing (Syntax Analysis)

- The interpreter checks the syntax using a parser.
- It builds an internal structure called an **Abstract Syntax Tree (AST)** that represents the code hierarchy.

4. Compilation to Bytecode

- Python compiles the AST to **bytecode**, a low-level, platform-independent representation.
- Stored in `.pyc` files inside the `__pycache__` directory.
- Bytecode is **not machine code**, but a format Python's Virtual Machine understands.

5. Execution by Python Virtual Machine (PVM)

- The **Python Virtual Machine (PVM)** takes the bytecode and **interprets it line-by-line**.
- This is where actual execution happens (calculating values, calling functions, etc.).